2025 – 2026

IBBDiS – Feedback & Display Interface

Realization Document

Alexandros Gkiorgkinis
Student Bachelor Applied Computer Science
Thomas More University of Applied Sciences
Internship Organization: European Commission – DG SCIC
February – May 2026

Glossary

Term / Abbreviation	Description
API	Application Programming Interface. A set of rules that allows two software systems to communicate with each other.
Cog	lightweight browser shell designed to display web content in full-screen kiosk mode on embedded devices.
Chipsee Panel	A brand of industrial touchscreen display hardware designed for continuous, always-on operation.
DG SCIC	Directorate-General for Interpretation. One of the departments of the European Commission responsible for multilingual communication and conference interpretation.
Horizon	An internal resource and room management system used at the European Commission.
IBBDiS	Interactive Building and Board Display System. A digital signage platform used to display information on meeting room panels.
Kiosk mode	A configuration in which a browser or application runs in full screen with no visible navigation controls, restricting the user to a single interface.
Mock data	Artificially created data used to simulate real system responses during development and testing.
PANDA	An internal event logging and data collection tool used within DG SCIC.
Raspberry Pi	A small, low-cost single-board computer commonly used in embedded and display systems.

SCIC GUID / av_system_id	A unique identifier used to link records across different internal systems.
WPEWebKit	An open-source web rendering engine optimised for embedded and low-resource devices.

Preface

This document is the realization document for my internship that I completed at the European Commission, in DG SCIC. A final part of my Applied Computer Science program that I need to complete at Thomas More. The internship took 14 weeks from February 23rd to May 22nd, 2026.

The project focused on extending IBBDiS an Internal Browser Based Digital Signage, by adding an interactive feedback collection system and a redesigned display interface for the meeting room panels and large entrance screens across EC buildings.

This report describes the full journey of the project: what the existing situation looked like, what was built and why, how the design evolved through feedback from colleagues, what challenges came up along the way, and what was ultimately achieved. I tried to write in a way, that everyone can read it, not only in a technical way.

I would like to thank my mentor and all the colleagues at DG SCIC who gave their time to review designs, answer technical questions, test prototypes, share their opinions and help me throughout the internship. Every piece of feedback whether it was good or remake it,created the final result.

Summary

This realization document shows the full flow of the 14 week internship at the European Commission in DG SCIC. The goal of the project was to extend the existing IBBDiS digital signage platform with a feedback collection system, that way meeting participants can rate their experience directly from the touchscreen panels installed outside the meeting rooms.

Before starting this internship, the panels only displayed basic information and there was no way to interact with it. With approximately 1,400 meeting rooms to manage, DG SCIC needed a way to collect structured feedback so they could track any issues from those rooms.

The project started with a frontend by designing and testing everything visually on my laptop then testing it on a real hardware. HTML prototypes were shared with colleagues from the very first week, and the design went through many rounds of feedback and improvements. The panels, the large entrance screens, and several advanced concepts were all redesigned and tested during this period.

By the end of the design phase, the main deliverables were complete: a three-state panel that shows the previous, ongoing, and upcoming meeting, a one-tap smiley feedback system from angriest to happiest, a room occupancy indicator, a redesigned datagrid template for large screens, and several advanced concepts including a day timeline, room booking from the panel, and an issue reporting button.

The backend part started in the second half of the internship. The right colleagues were identified for it, mapped out the API fields that were needed, and a mock data system was built for a demonstration on how it would work. Some access issues slowed down the progress, but everything was prepared and ready to implement once access was resolved.

Table of Contents

1. Introduction

In this chapter, it explains where the internship took place, what problem it was trying to solve, what the goals were, how the work was approached, and what was achieved by the end.

1.1 Where the Internship Took Place

The internship was done at the European Commission, in the department DG SCIC (Directorate-General for Interpretation). They are responsible for interpretation services for EU institutions and taking care of all the conference and meeting facilities across the EC buildings in Brussels.

The department handles a large number of meeting rooms of different sizes and purposes. To make sure people walking past a room always know what is happening inside, DG SCIC uses a digital signage system called IBBDiS (Internal Browser Based Digital Signage). Small screens are mounted outside each room and display real-time meeting information pulled from internal EC systems.

There are three types of meeting rooms. Type A rooms are the smallest rooms. Type B rooms are medium-sized and equipped with more sophisticated setups. Type C rooms are the largest, used for major conferences and events, and they have interpreters inside booths. The type C rooms were previously equipped with older screens called AMX which had a simple smiley buttons for ratings at the end of each meetings, but those screens are being decommissioned and replaced.

1.2 The Problem

The core problem is that the existing IBBDiS system only shows information. It cannot receive any information from the people. There is not a way to know if a meeting participant had a good or bad experience, whether the equipment worked smoothly, or whether something was wrong with the room.

This matters more than it might seem but DG SCIC manages 1,400 meeting rooms worldwide. When something goes wrong in a room like a broken projector, bad lighting, a temperature issue, no one really knows unless someone physically walks up to them and reports it. By the time it gets noticed and fixed, the same problem may have affected multiple meetings in that room.

1.3 What the Project Set Out to Do

The goal of this project was to add the missing piece: a way for meeting participants to quickly and easily rate their experience directly from the panel outside the room. The interaction had to be as simple as possible a one tap on a smiley face because if it takes more than a second, people will not bother.

Beyond the feedback system, the project also covered redesigning the display interface itself, as it only showed two states (ongoing and upcoming meeting) and needed a third (the previous meeting) to give a more complete picture of the room's schedule. Large entrance screens showing the full day's meeting list across the building also needed a visual refresh. And everything had to work for people with color vision deficiency, not just standard vision.

The full list of deliverables was:

- A redesigned meeting room panel template showing previous, ongoing, and upcoming meetings
- A one-tap smiley feedback system with five rating levels
- A room occupancy indicator showing whether the room is available, booked, or occupied
- A redesigned datagrid template for the large screens at building entrances and floor levels
- A non-interactive version of the template for TV-sized screens that have no touch capability
- Accessibility compliance across all designs, including support for color vision deficiency
- Backend integration to store and process the feedback data collected
- Follow the EC color code
- Full documentation of design decisions and the reasoning behind them

1.4 How It Was Approached

The development started with the frontend. Instead of worrying about the backend straight away, the focus was on getting the design right first by building it, testing it on real hardware, and then improving it based on the feedbacks and having inspiration on google from existing products in the industry.
Each week, new prototypes were shared about the ideas, the designs that were combined, everything was shared through Teams group and in person at the office. Each would give their feedback and help me with what it's missing, then keep updating it and keep sharing it around. Making this loop over again.
Afterwards, with those templates, they were used to test them right through the IBBDIs panel which helped me to notice smalls details that you simply cannot see in a browser like a small spacing, font sizes, colors, and how the touch interaction actually felt.
The mentor advised documenting all decisions made throughout the process, not just the final result, but also about the process on how it was achieved and why those changes were kept.

1.5 What Was Achieved

By the time this document was written, the main visual templated was designed, tested on real hardware, reviewed by colleagues, and refined through multiple rounds of feedback. The datagrid template that displays all the meeting of the building was already live on a test screen and waiting for approval.

The backend integration phase was underway. The data fields needed from the Horizon API were mapped out, a mock data object was built into the prototype to demonstrate all four room states, and feedback data was being stored locally in the browser to show what would be sent to the backend. API access trying to be solved through the IT helpdesk and colleagues, and a meeting with the responsible development team was being planned to figure out the next steps.

2. Analysis

Before building anything, it was important to understand the existing system like how it works, what it is built on, and what is already there. This chapter describes that analysis.

2.1 The Existing System — IBBDiS

IBBDiS stands for Internal Browser Based Digital Signage. It is a platform built and maintained by DG SCIC that displays meeting information on screens across EC buildings. The way it works, IBBDiS pushes HTML template files to each Raspberry Pi device. The device uses a browser called WPEWebKit through a shell called Cog, which displays the template in full-screen with no address bar, no navigation, just the template filling the entire screen. The template is an HTML file that uses JavaScript to poll internal API endpoints, fetch the latest room and meeting data and update the display automatically. This means uploading a template is as simple as deploying a new HTML file. The system pulls meeting information from internal EC systems and sends it to the screens. Each screen shows the room name, the current time and date, the ongoing and upcoming meeting.

The larger screens at building entrances and floor levels show a full table of all the meetings happening across the building for that day.

2.2 The Hardware

The small screens outside meeting rooms are Chipsee 10-inch touchscreen panels. Each one runs on Raspberry Pi, basically a small pc that sits behind the screen and handles everything. The Raspberry Pi runs the browser software and shows the HTML template. It connects through the EC network using an Ethernet cable, which also powers the device, that helps installation tidy with just one cable.

The specific model used and tested was the AIO-CM4-101, a 10.1-inch chipsee device.

For the larger entrance and floor screens, a separate Raspberry Pi is connected to a bigger monitor via HDMI. Some spots also have TV-sized screens running the same templates but without any touch functionality.

2.3 The Data Flow

IBBDiS does not store meeting data itself. It pulls information from internal EC systems, that handle the meeting rooms bookings and AV equipment and formats it for display on the screen. The system that is responsible for live room status is called Horizon, which is part of a larger platform called Merase. Horizon tracks live signals from sensors and AV equipment to determine whenever the room is occupied or not. The fields that are extracted for the rooms from the Horizon API are: occupied (when someone is detected in the room), booked (when a meeting is reserved), online (when the panel device is connected and active), and av_system_id (the unique identifier that links the room in Horizon with the same room in IBBDiS). For the feedback, it is sent to PANDA and contains a timestamp, an event type identifier, a rating from 1 to 5, and a room identifier, that way every feedback is tracked to the exact room and time it was submitted.

The three systems work together as part of MERASE, an ecosystem built and maintained by DG SCIC. The software that runs each meeting room is called SharP. SharP connects to the AV equipment, manages energy saving, and detects room presence. Horizon remotely connects to every SharP enabled room and exposes the data through an API, this is how the panel retrieves the live room information. Panda is more like the master data for all meeting space usage across the EC. It handles the configuration, the deployment and stores the logs, which is why it was chosen to be the destination for the smiley feedback.

2.4 Tools and Technologies Used

The following tools and technologies that were used are:

- **HTML, CSS, and JavaScript.** Everything was built as standalone HTML files running directly in the browser. The mockup file provided by the department was used as a starting point.

- **Microsoft Teams:** The primary channel for all communication and collaboration.
- **Horizon API:** Used to retrieve live room status data. Getting access to it turned out to be a big challenge.
- **Color Blindness Simulation Tools:** Online tools used to verify designs for people with different types of color vision deficiency.
- Chipsee: The 10-inch touchscreen used for testing.
- Raspberry Pi: The small computer that runs the display software and renders the templates.

2.5 Technical Considerations and Challenges

When the template with the smiley feedback was tested on the panel, tapping on the smileys was not working but on laptop it worked perfectly. After, figuring out the issue, it turned out that the WPEWebKit which is the browser that runs on a raspberry Pi, does not fire the click event reliably on a touch input, but switching to touchstart solved the issue.
The panel displays a real-time progress bar to show how far the current meeting is. This was implemented in JavaScript by converting the start time, end time and current time into minutes, calculating the percentage of the meeting already elapsed, and applying to the width of the bar. The bar updates automatically every second.
Another consideration was the size of the screen. The chipsee is only 10 inches, which meant fitting the whole design with the previous, ongoing and upcoming meeting sections with the smiley feedback system required a careful balancing. Adjusting one thing would break the other usually, for example increasing the font size of the meeting titles would push the feedback smiley section out of view, while reducing it too much would make the most important information on the screen hard to read. Finding the right balance between elements took a lot of effort.

2.6 How Design Decisions Were Made

From the start of the internship, the mentor made it clear to document the entire process. What thought and decision, the feedbacks, the changes and most important the why.

Each time a new design was ready, it was shared with colleagues in Teams or in person, collected their reactions or opinions, updated the design, and shared it again. Every time a decision was made to change something, a note was taken of the reason. Over time this created a clear record of how the design evolved that anyone could look back at and understand the why.

What was interesting, is that how many different opinions created the final result. Feedbacks came from the mentor, from technical colleagues, from the graphics team, from industry examples, from my own ideas and from colleagues that were encountered into the office. Every conversation added something and the designs were upgraded step by step.

3. Meeting Room Panel Template

This chapter covers the full design journey of the meeting room panel template, from the original version I received at the start of the internship all the way to the final design.

3.1 Starting Point — The Original Template

At the start of the internship, an HTML mockup file was provided from what was used outside the meeting rooms. The left side of the template was showing the EC logo, a clock, and the date, and on the right side was showing the room name at the top, the current or next meeting in the middle, and the upcoming meeting or nothing at the bottom. Whenever there was no meeting it was displaying No meetings in this current time.
With the file, a short PDF documentation was also provided explaining how to change the fake data in the code, which would change what appeared on screen. This helped me understand the different states the screen could be in an ongoing meeting, a meeting coming up soon, or no more meetings for the day.
The mockup file really helped me but there were 2 issues: it only showed two meeting states and there was no way to interact with it.

Figure 1: Original IBDDiS panel template received at the start of the internship.

3.2 First Prototype and Early Feedback

Within the first few days, a first prototype was built on top of the original template. A pulsing yellow button was added that appeared during or after a meeting, holding it down for half a second opened a full-screen overlay with five smiley faces and a countdown timer. After tapping a smiley a thank you message appeared and it closed. A screensaver was also added that appeared after two minutes of inactivity, showing a bouncing EC logo to protect the screen from burn-in pixels.
The prototype was shared in the Teams group and received feedback really fast. Three main things came back: the screensaver looked like a crashed screen and confused people, holding a button for half a second before tapping a smiley was too many steps, and the layout needed a retouch. Someone also suggested adding a third state to show the previous meeting alongside the current and upcoming ones. Good feedback to work with. The screensaver was removed entirely because of two reasons. After the first round of feedback, the screensaver panel would look dark and broken in a busy corridor. Then, colleagues responsible for the panels pointed out that they are designed to run 24/7 continuously, meaning burn-in protection was never needed in the first place, the screensaver was added based on an assumption which turned out to be wrong.The hold mechanism was replaced in favour of a direct tap, and made the three-state layout the new goal.

Figure 2: First prototype after the first round of feedback, with the Rate this session button

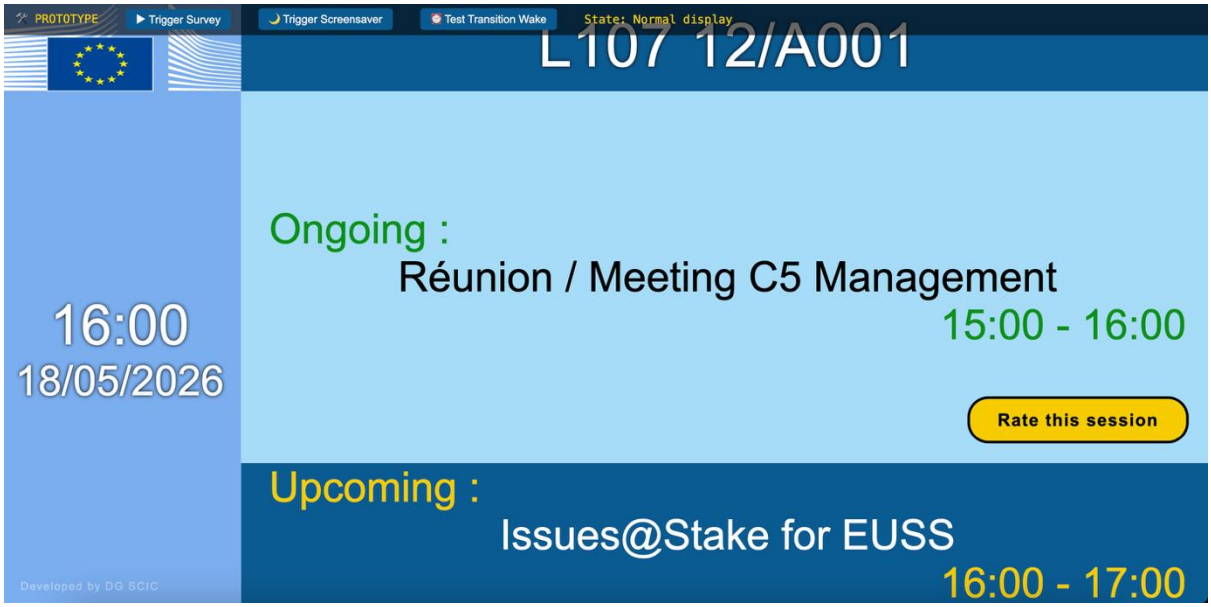


Figure 3: Feedback overlay with smiley rating options

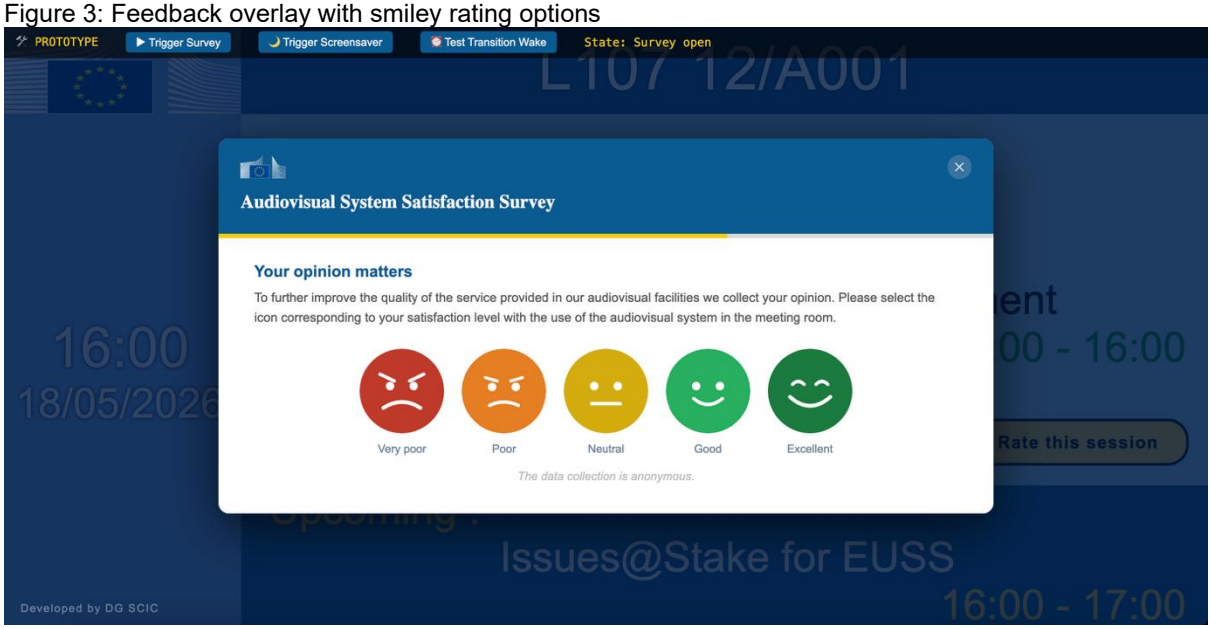


Figure 4: First screensaver design, bouncing EC logo, mistaken for a broken panel.

Figure 5: Redesigned screensaver, later removed as the panels are built for 24/7 operation.

3.3 Finding the Right Layout

Following the first round of feedback, several layout options were designed and shared them in the Teams group for people to vote on. Over about a week colleagues compared them, shared their thoughts, and a clear favourite emerged the third layout out of four.
What worked about it was the reading direction. Normally people are reading all the way from top to bottom and then from left to right, so by adding the previous meeting at the top, the ongoing meeting in the middle, and the upcoming meeting at the bottom just felt normal. Nobody needed an explanation.
Colors also came up a lot during this period. Should the smileys go from red to green? Should green indicate an ongoing meeting? Opinions were split, some colleagues liked the colors, others found them noisy. After going back and forth several times the decision was to follow the EC color palette as the main guide, keeping things consistent with the institution's visual style.
Figure 6: Poll results from the Teams group

Figure 7: Layout 1 shared for colleague voting

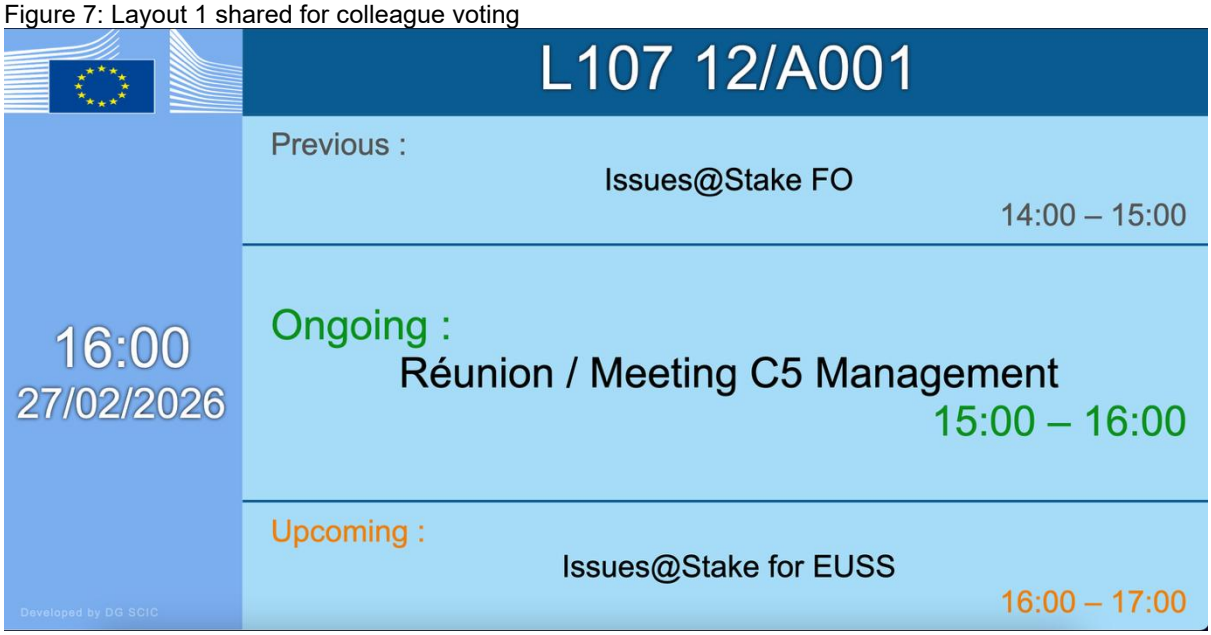


Figure 8: Layout 2 shared for colleague voting

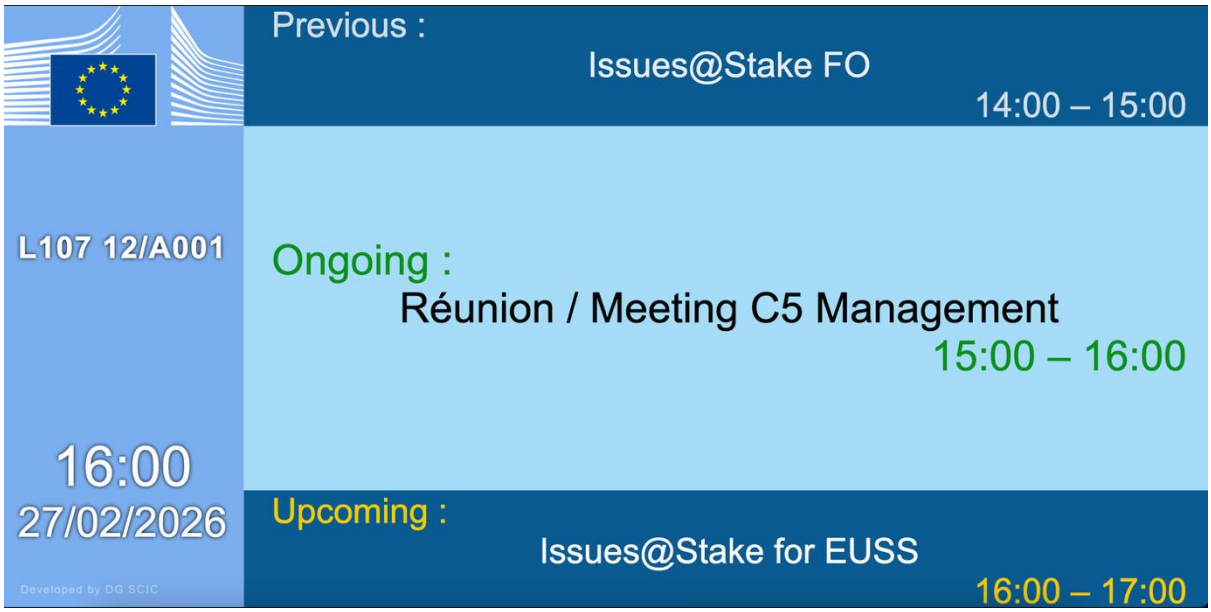


Figure 9: Layout 3 shared for colleague voting

Figure 10: Layout 4 shared for colleague voting

3.4 Accessibility — A Wake-Up Call

Partway through the design process a colleague posted a question in the Teams group that stopped me in my tracks: did the design comply with WCAG accessibility guidelines and did it support people with color vision deficiency?

Honestly, it had not been considered up to that point. The templates were run through color blindness simulation tools and it became clear pretty quickly that using only red and green to show room status was a real problem, a lot of people cannot tell those two colors apart.

So, the occupancy indicator was redesigned. Instead of relying only on the bar color, a small dot that pulses was added. Next to the dot a text showing the current state: Available, Booked, or Occupied was added. That way the color gives a quick visual for most of people, but the text makes sure everyone understands regardless of their vision.

Accessibility should have been thought from the beginning, not something that needed to be mentioned by a colleague. From that point on, every design was tested through the simulation tools before being shared, a habit that continued for the rest of the internship.

Figure 11: Protanopia simulation (no red)

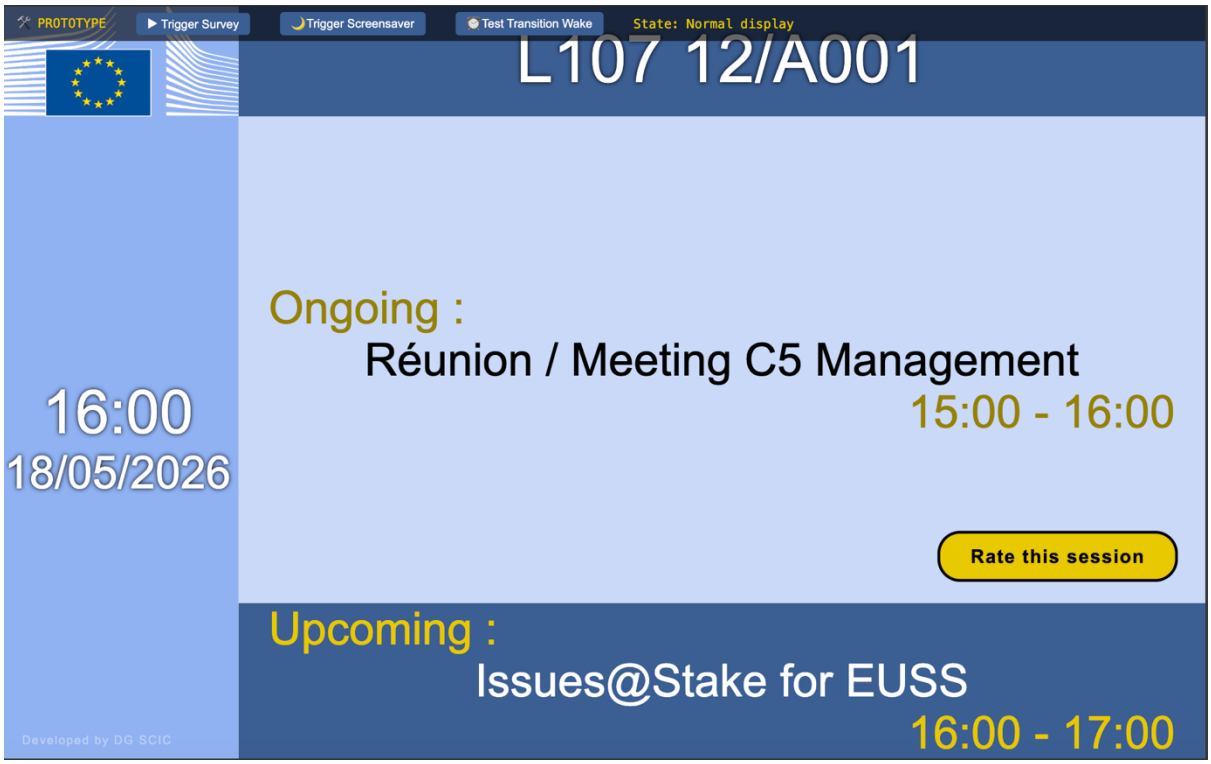


Figure 12: Deuteranopia simulation (no green)

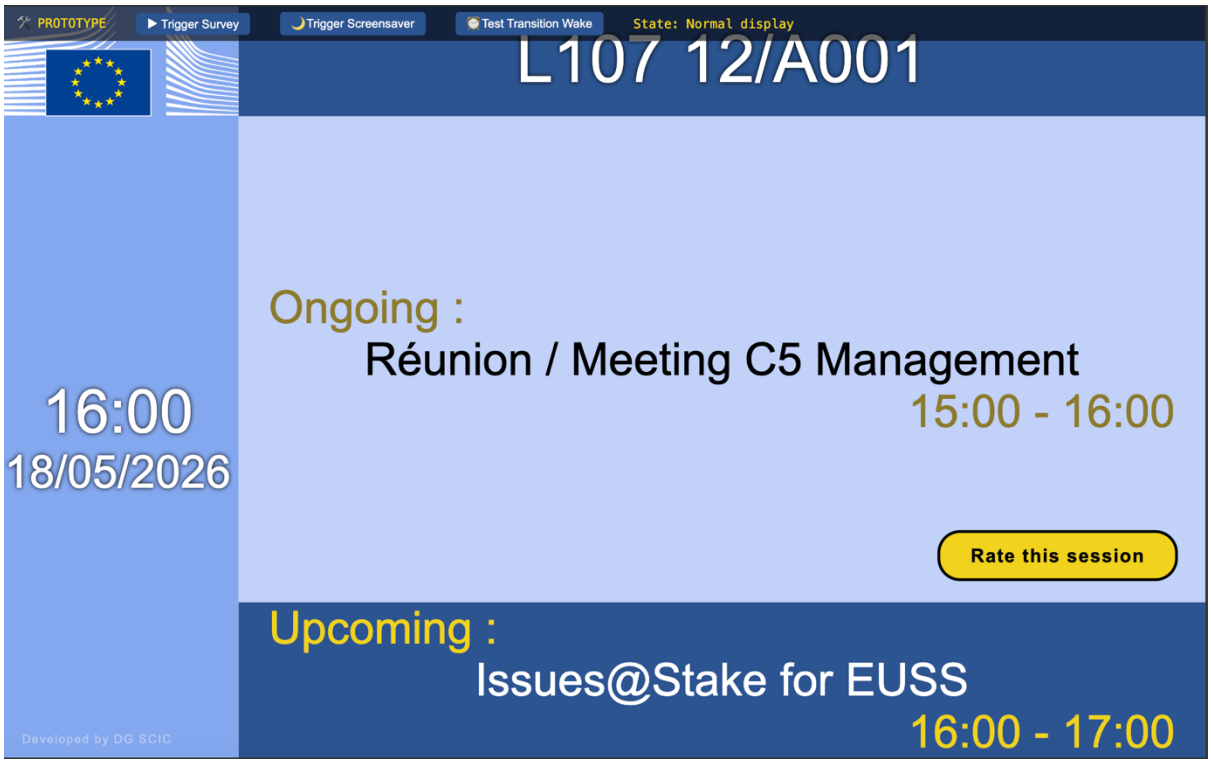


Figure 13: Tritanopia simulation (no blue)

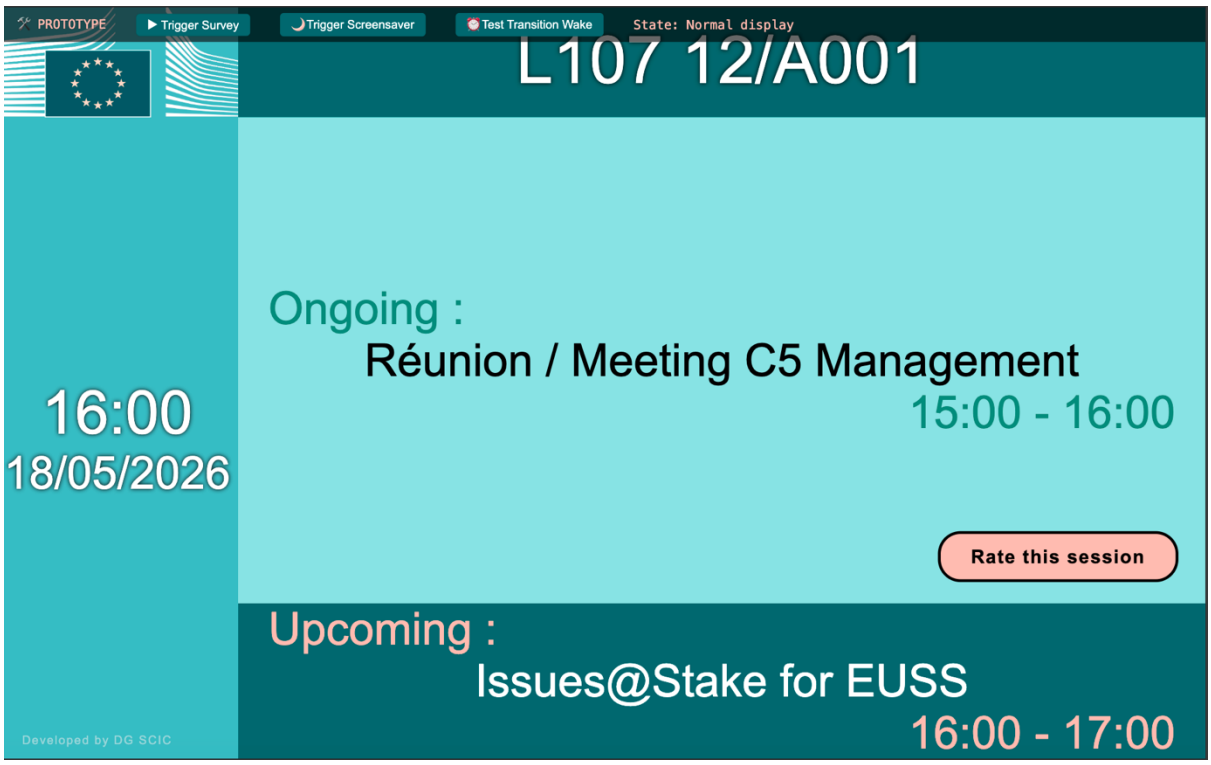
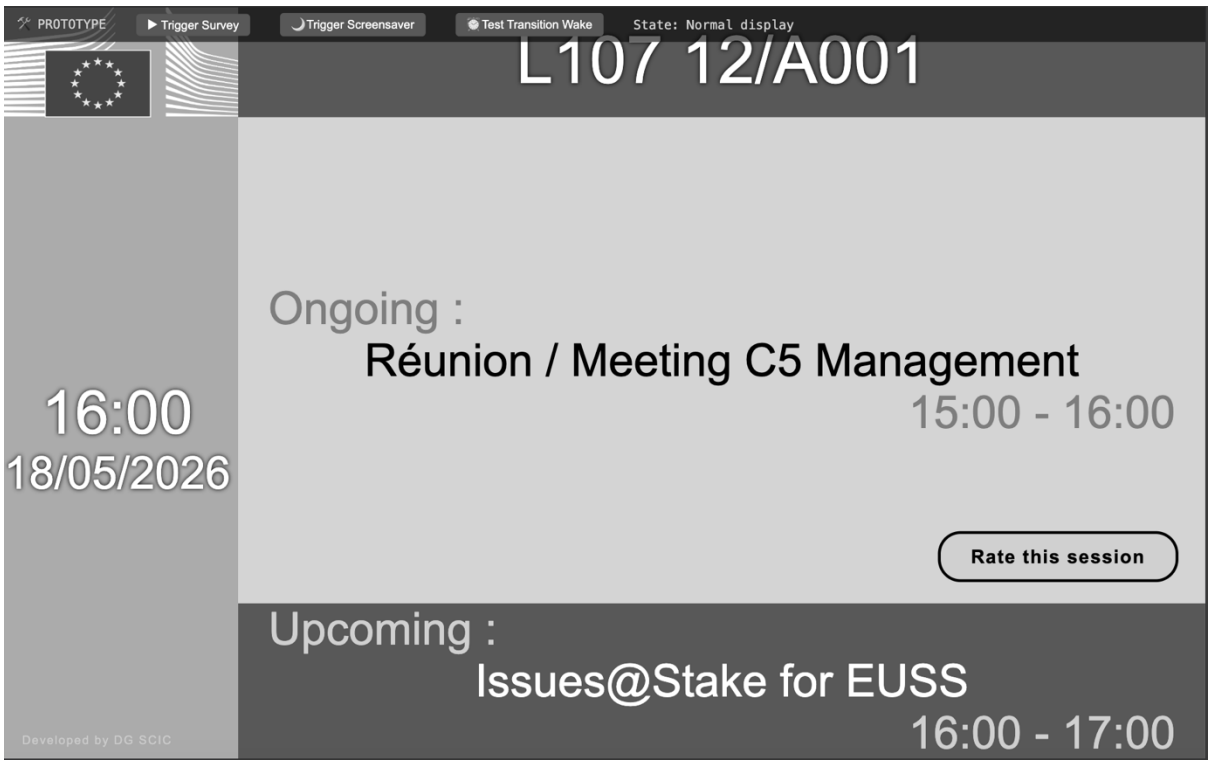


Figure 14: Achromatopsia simulation (no colors)



3.5 The Occupancy Indicator

What started as a simple colored bar at the top of the screen, became one of the most iterated elements of the entire project. The first approach used only colors, green for available, orange for booked and red for occupied which worked for the time being but still left two states unclear: a room that is booked but physically empty and a room that is available but someone inside. Just because a meeting was reserved does not mean that someone is actually present.
Following the accessibility feedback from the section 3.4, a pulsing dot and a text label were added to communicate the state through both color and text at the same time. This version had four named states: free, booked, busy and unbooked.

Figure 15: occupancy state: Free

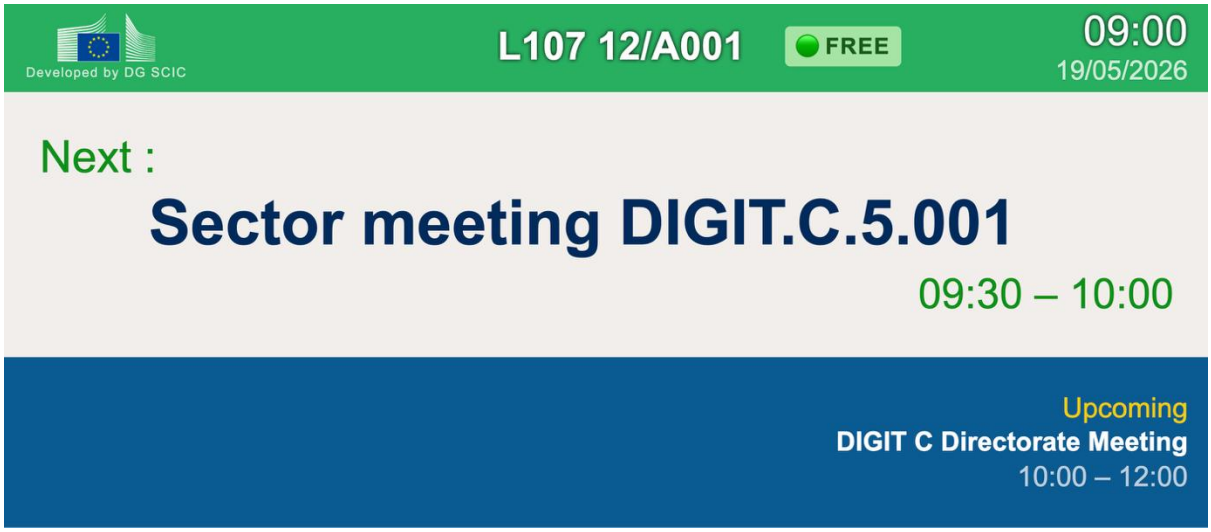


Figure 16: occupancy state: Booked

Figure 17: occupancy state: Busy

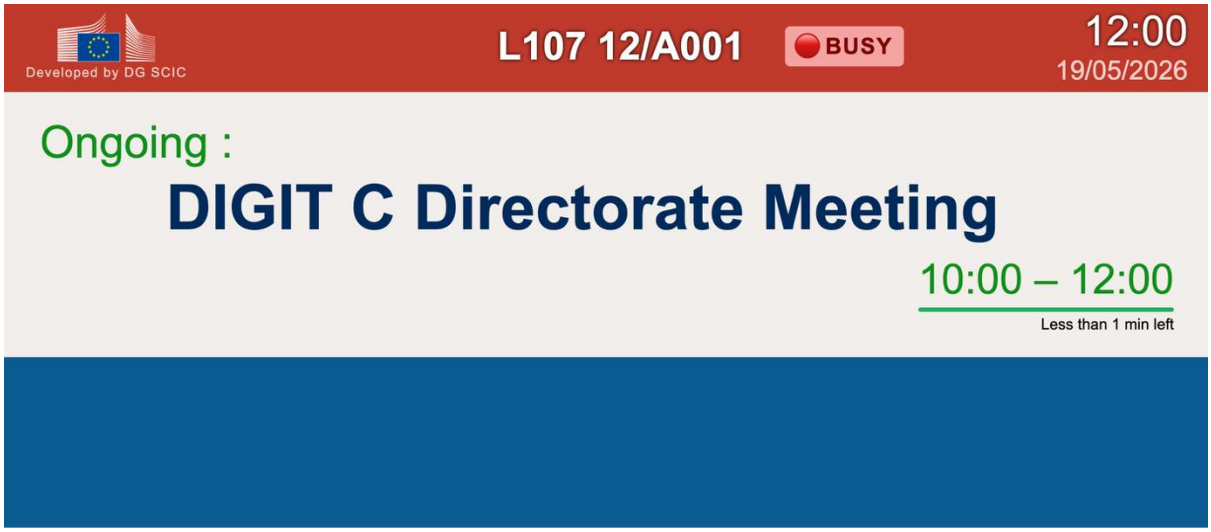
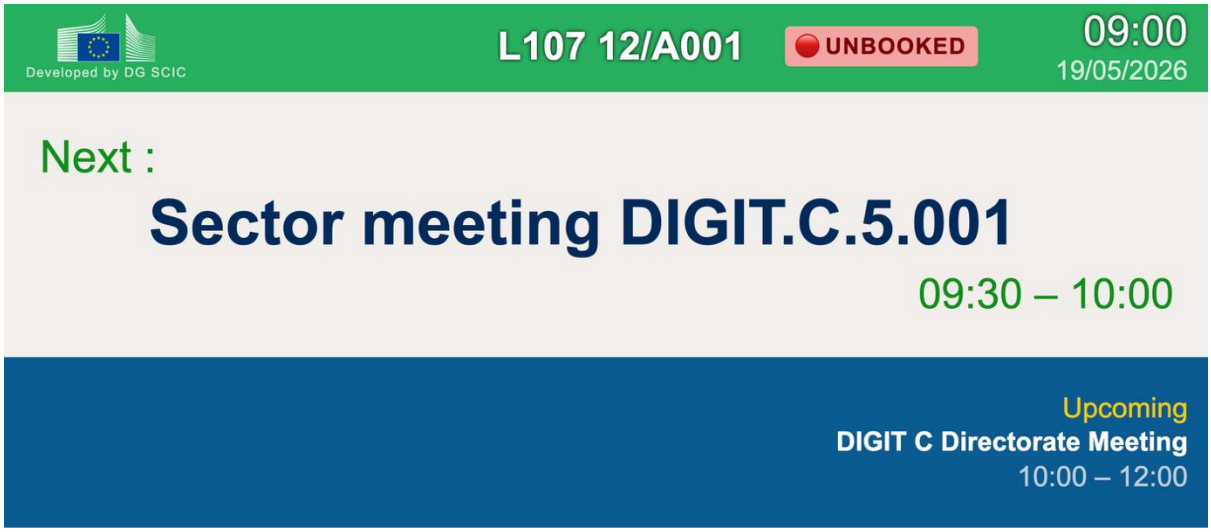


Figure 18: occupancy state: Unbooked



However, colleagues mentioned that the two states are confusing even with the labels. What seemed like a simple status display turned out to be a genuinely difficult design problem. How do you communicate four room states at a glance, without explanation, to anyone walking past? The lesson here was that the simpler something looks, the harder it usually was to design. After, going through rounds of feedbacks the text badges with the pulsing dot were removed entirely to reduce clutter, a person icon was added with a short label mentioning either “In Room” or “Empty”. An icon communicates presence without requiring any words, it is understood immediately. Removing the dot and the badge color reduced the visual clutter and made the layout feel cleaner. To strengthen the visual connection, the meeting label color was matching with the color of the top bar. So, orange when booked, red when occupied. This way the states were communicated clearly: the bar color, the label color, and the presence icon. The final result of this is visible in the section 3.9.

3.6 The Smiley Feedback System

The feedback system went through a few changes. The original idea had a button that needed to be held down to open the survey. From the feedback they said that this was too complicated, so the interaction was simplified to a direct tap.
After tapping a smiley, a thank you message appears for a couple of seconds and then the screen resets.
A small interaction detail was implemented: a smiley is hidden, so that the thank you message pop up could fit.
A progress bar was also added below the meeting title to show how far the current meeting is and how much time is left.

Figure 19: Panel design showing feedback smileys and meeting progress bar

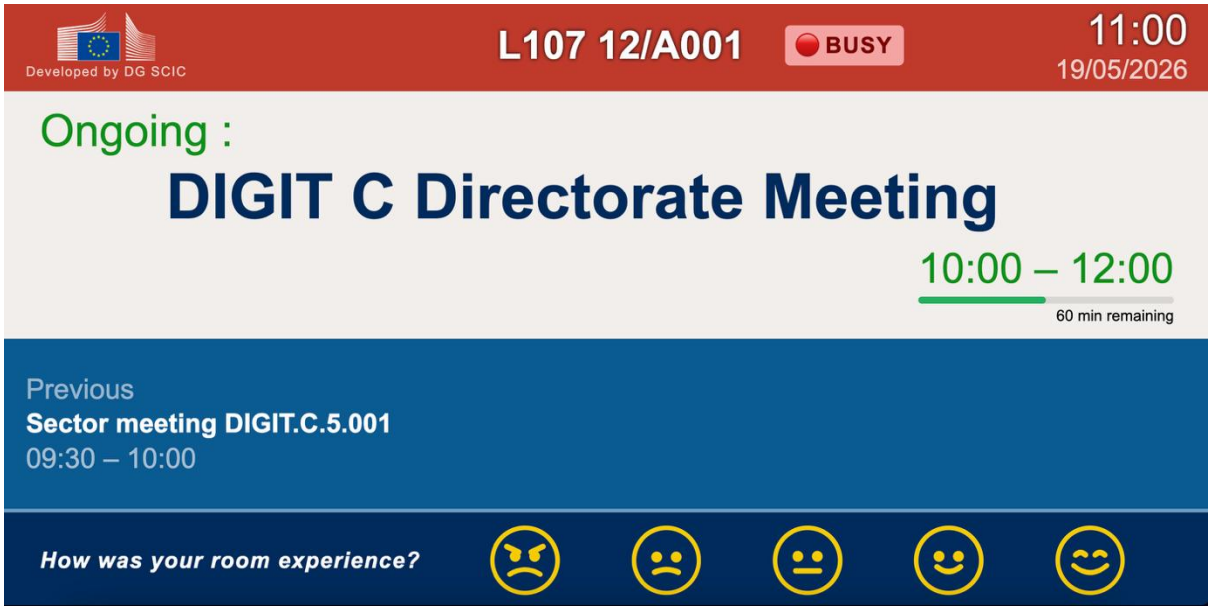


Figure 20: Feedback interaction, smiley selected with confirmation message

3.7 Advanced Design Concepts

Inspired by existing products from the companies Crestron and AskCody, two advanced concepts designs were created. The first uses a horizontal timeline from 08:00 to 18:00 showing the free timeslot in green and booked timeslots in red. The second concept adds the organiser's name, a starts-in countdown, and a Report an Issue button that sends a support ticket directly from the panel.
Both were well received but a colleague pointed out they have many technical dependencies. The basic feedback survey needs to be approved first before moving to these.

Figure 21: Inspiration template concept from Crestron

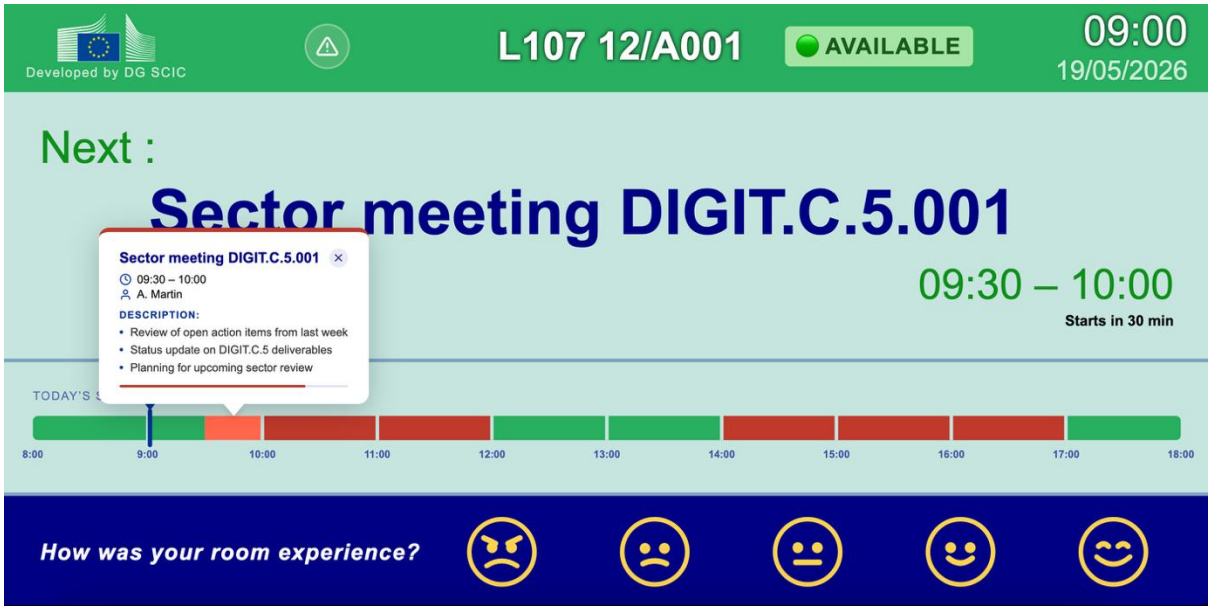


Figure 22: Inspiration template concept from AskCody

3.8 Input from the Graphics Team

The OIB graphics team (Office for Infrastructure and Logistics) are responsible for the visual standards and design guidelines across EC buildings. A call was made with two members of the team, to get their professional design opinions of the latest design and what they think of the panel template. The feedback was overall positive, they were surprised about the design especially not knowing anything about design and being an IT student. Some specific elements were they suggested to make the long meeting titles wrap to two lines with three dots if the text was still too long, rather than have it a automatic scroll effect. They also recommended adding colors to the smiley faces. The eyebrows were removed from the angriest smiley as they made it look too aggressive. The meeting titles were aligned to the left and a divider line was added to keep the layout balanced when only one meeting state was visible. When showing also the new concepts inspired by AskCody and Crestron, they raised a GDPR consideration regarding displaying the organiser's name on the panel.

3.9 Final Design

The final design was the result of more than twenty rounds of feedback from colleagues, the mentor, and the OIB graphics team all together. The panel shows three meeting states: the previous meeting, the ongoing meeting with a progress bar and time remaining, and the upcoming meeting. The occupancy is displayed at the top with the bar color and a presence icon showing either "In Room" or "Empty". Five colored smileys are always visible at the bottom for a quick feedback. Long meeting titles wrap to two lines with three dots if needed. The label color matches the top bar color to create a clear visual connection between the booking state and the meeting information. The template follows the EC color palette and was verified for color vision deficiency compliance. A non-interactive TV version was also created for the larger non-touch screens across the buildings. Since those screens have no touch capability, the smiley feedback section was removed and the layout was made bigger and responsive to adapt different large screens.

Figure 23: Final Design – room available

Figure 24: Final Design – not booked, but someone inside

Figure 25: Final Design – booked, no one inside

Figure 26: Final Design – booked, someone inside

Figure 27: TV version displayed on large non-touch screen

4. Datagrid Template

Alongside the panel template, the large datagrid template was also redesigned, shown at building entrances and on each floor. This template displays a full table of the day's meetings across the building.

4.1 The Original Template

The original was a dense HTML table with room name, meeting title, start time, end time, and platform codes like (W) for Webex or (T) for Teams in parentheses. It worked but was heavy to read and had no visual way to show which meetings were ongoing or finished.

Figure 28: Original datagrid template

4.2 The Redesign

The start and end times were merged into a single column with a real-time progress bar in between. The platform codes were removed as they were being phased out and replaced them with three small badges: one for live webstreaming, one for audio recording, and one for videoconferencing.

Figure 29: Datagrid with icon legend — final version

4.3 Technical Consideration

The datagrid template runs also on a raspberry Pi hardware and WPEWebkit browser as the meeting room panels. The feature that was used in this template was a real-time progress bar that showed the start and end time of the meetings. It was calculating the percentage of the meetings that were ongoing in JavaScript, by comparing the current time against the start and end time and converting the result in a width of the progress bar. Whenever the meeting was over a check symbol was added next to it and whenever the meeting did not start the progress bar was not visible. The badges for the live web streaming, audio recording, and videoconferencing were built in a simple HTML span to keep it lightweight and make it show fast. The template needed to be dynamic to different sizes because all the screens are not the same size.

4.4 Status

The redesigned template was sent to a colleague who integrated it into the IBBDiS test environment. It is currently live on a test screen and waiting for approval before uploading it across all buildings.

5. Backend Integration

The second half of the internship focused on starting the backend integration, connecting the prototype to live data.

5.1 Understanding the Data Needed

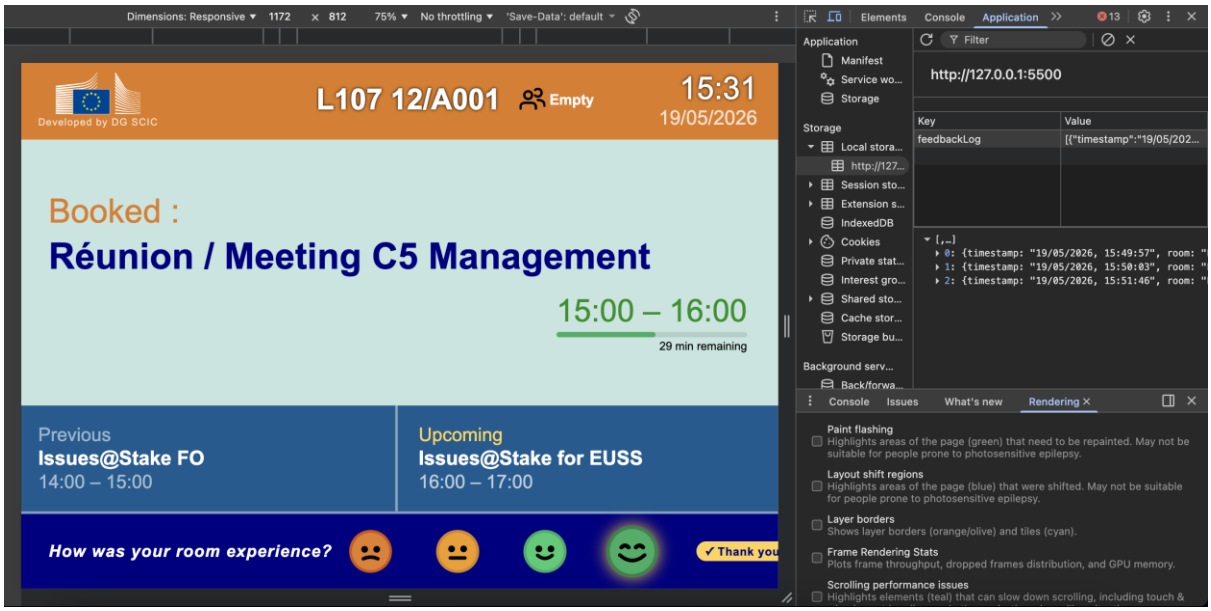
Two types of data are needed: room status from the Horizon API (occupied, booked, available) and a way to submit and store the smiley feedback linked to a specific room. The room identifier used by IBBDiS and Horizon is the SCIC GUID, which connects both systems. From the Horizon API, the fields that are really used are occupied (when someone is in the room), booked (when a meeting is scheduled), online (when a panel is connected) and av_system_id (the unique identifier used to query the API). For the feedback side, each event is sent through PANDA, which stores a timestamp, an event type, a rating (1-5) and the room identifier. That way each rating is linked to the specific room and the time it was submitted.

PANDA is the management layer of the MERASE ecosystem, designed to act as a master data source for SCIC spaces. It collects logs and usage statistics from every room in the network. Sending the smiley feedback as a structured event to PANDA, each tap becomes a log event, with a timestamp, linked to the room identifier.

5.2 Demonstrating the Flow Without Live Access

Since API access was not available yet, the JSON fields of the API were provided and a ROOM_DATA object was built into the prototype that can be used to manually toggle the room states, to show how the screen looks in each situation. Feedback data is saved to the browser's local storage on each tap, showing exactly what would be sent to the backend.

Figure: 30: Feedback data stored in localStorage during testing



The ROOM_DATA is used in the template to demonstrate exactly the fields that the real Horizon API would return. By changing the occupancy and booked values manually, every room state can be tested without a live connection. When the real API will be available, it will be replaced by a live fetch call, the rest of the template logic stays the same.

```
var ROOM_DATA = {
      av_system_id: "d5fdf257-a942-2990-aa34-95c55cbfb9af",
      power_state:  "on",
      occupancy:    "free",
      booked:       "booked"
   };
```

The template uses these two fields to calculate which of the four room states to display. The logic combines physical occupancy with calendar booking to produce the correct result:

```
if (ROOM_DATA.occupancy === "in_use" && effectiveBooked === "booked") {
   // occupied: someone is physically in a booked room
} else if (ROOM_DATA.occupancy === "in_use" && effectiveBooked !== "booked") {
   // unbooked: someone is in the room but no meeting scheduled
} else if (effectiveBooked === "booked") {
   // booked: meeting scheduled but room is empty
} else {
   // free: no one there, nothing scheduled
}
```

When a smiley is tapped, the feedback is saved with the room identifier, the current meeting title, and a timestamp, matching exactly what would be sent to PANDA in a live implementation:

```
function saveFeedback(rating) {
   var entry = {
      timestamp: new Date().toLocaleString(),
      room: ROOM_NAME,
      rating: rating,
      av_system_id: ROOM_DATA.av_system_id
   };
   localStorage.setItem("feedbackLog", JSON.stringify(log));
}
```

To go a bit further, a local mock server called SignageTest was built using ASP.NET Core. It demonstrates three API endpoints that mirrors the real EC systems:

- GET /api/horizon/room-info/{id} returns the room state for a given av_system_id, with toggleable occupied and booked fields
- POST /api/panda/event_logger receives a feedback event (timestamp, event type, rating, room ID) and stores it in memory
- GET /api/booking/{id} returns the meeting list for a room

This allowed to show the complete data flow: the template fetches room data, displays the correct state, and when a smiley is tapped, the feedback event is sent to the Mock PANDA endpoint and stored. The demo proves the full concept works technically, once real API access is granted, only the base URL needs to change

5.3 Access Challenges

Getting access to the Horizon API was more complicated than expected. As an external intern on a personal laptop, it was not possible to connect directly to the EC network where the API endpoints were needed, but only as a Guest network. The IT helpdesk tried to help and find a solution managing to set up the EC email account but the internal network endpoints were still out of reach. What made it harder wasn't only about not having access to it, but midway through, the head of unit for another department paused the project as her team had not been informed about it, which led to colleagues not being able to help, they needed to follow the decision of the management. The mentor's guidance was to continue the project with what was available. It was frustrating at the time, but it was also a good example of how a large institution actually works. The progress does not just depend on technical skills but on approvals, communication and decisions made by people who may not be directly involved in the work.

The API access should have been submitted in the very first week of the internship, not something to start once the frontend was done. In the European Commission, access to internal systems goes through official approval channels that can take weeks or months and are outside intern's control. Starting earlier would not have guaranteed access, but it would have given more time to find a solution.

5.4 Next Steps

A working demonstration of the backend was created as a local test application. It shows the Horizon API endpoint that returns room status by av_system_id, and PANDA endpoint that receives the feedback events with a timestamp, event type, and event data fields. The next steps are to make the local demo live. The HTML file needs to be connected with the real Horizon API instead of the mock one, using the av_system_id to fetch the live status. The feedback POST call needs to redirect from the test PANDA to the production event_logger on the actual PANDA. Since the panels are powered on through the EC network, the endpoints need to be accessible from within that network and have a proper authorization. Once access is granted and connected, it will need a final testing on a real hardware before full deployment.

The choice to save the feedback through PANDA rather than storing it inside IBBDiS was also architecturally consistent: PANDA is the master data layer of the MERASE ecosystem, and IBBDiS is purely a display platform. Keeping them separate means the feedback data lives where the rest of the room usage data already lives in the system designed to manage it.

6. Conclusion

6.1 What Was Set Out to Do

The goal at the start of the internship was to make the IBBDiS interactive for the first time. Adding an interactive feedback system so meeting participants could rate their experience directly from the panel, and a redesigned display interface that showed a clearer picture of the room schedule. Both the panel template and the datagrid template needed to be accessible, follow the EC color code and work reliably on the screens. Before this project, the panels were a one-way system: they displayed information but had no way to receive any input from the people walking past.

6.2 What Was Achieved

All the deliverables that were completed during the internship are a fully redesigned meeting room panel template, tested on real hardware, reviewed by colleagues, mentor and the OIB graphics team which was updated multiple times through their feedbacks. A non-interactive TV version was also created for larger non-touch screens. The datagrid template for building entrances that shows all the meetings of the current building was redesigned and waiting for approval to be deployed across all buildings. A mock data system of the meeting panel was created for a demonstration purpose and show exactly how the backend integration would work. Feedback is stored locally. All designs were verified for color vision deficiency and followed the EC color code.

6.3 What Remains

The main remaining work is the backend integration: connecting the prototype to live Horizon data, using the real av_system_id, and redirecting the feedback POST from local mock to the production PANDA event_logger endpoint. To do this, the panels need to have network access to those endpoints from the EC network and a proper authorization. Once obtaining that access, a final round of testing needs to be done before uploading it across the rooms.

6.4 Reflections

This internship was definitely different from anything experienced in a classroom. The environment, the pace, the complexity of working inside a real institution like the European Commission were things that could not have been prepared for in advance. The biggest surprise was how difficult something looking simple could turn out to be. The occupancy indicator showing whenever a room is free or not went through more rounds of feedback than any other element and was still being refined in the last weeks of the internship. Every person who looked at it saw it differently. That taught me that in a real project, designs are never about what looks good mostly but more about what makes sense to the people who actually use it every day. Onae thing that made me see the whole project differently was during a conversation with my mentor towards the end of the internship. I was confused and frustrated about not having access to API and be able to test it with real data, he told me that my job was not to develop everything from scratch but to analyze the problem, think it through, and propose specific solutions. That perspective made me step back and a lot of pressure took off and made me realize how much had actually been achieved for the European Commission. If I could do it again, I would start the API access process in the beginning of the weeks rather than wait as a next step once the front end was done. This internship changed the way I approach the development. I now think more carefully about the people who will use what I build, test on a hardware as early as possible, the real challenge in a large institution is navigating the people, the approvals and the constraints around it.

Looking back, the most important skill this internship developed was not just technical skill. It was learning how to work inside a large institution where every decision depends on multiple teams, approval processes, and sometimes conflicting priorities. The code and the design were the straightforward part. Understanding who needs to be informed, who needs to approve, and in what order, turned out to be just as important. That is something a classroom project cannot really teach but more of an experience.

6.5 Recommendations

From the experience gained during the internship, the recommendations are made for anyone continuing this project or working something similar in the EC. Test the templates on a real hardware as soon as

possible. Some issues only became visible on the panel and would not be noticeable in a browser alone. By testing it on a panel it avoids having to fix things later in the process. Document everything, having clear notes of what was made, the feedbacks, why decisions were made is what makes the design process legible to anyone reading later. Start the API access process early, gaining access to the EC internal network as an external intern requires specific authorizations that takes time but you might not be able to access at all. The foundation is already in place, the designs are already tested and approved. Once the access barrier is resolved, the feedback system can go live quickly.

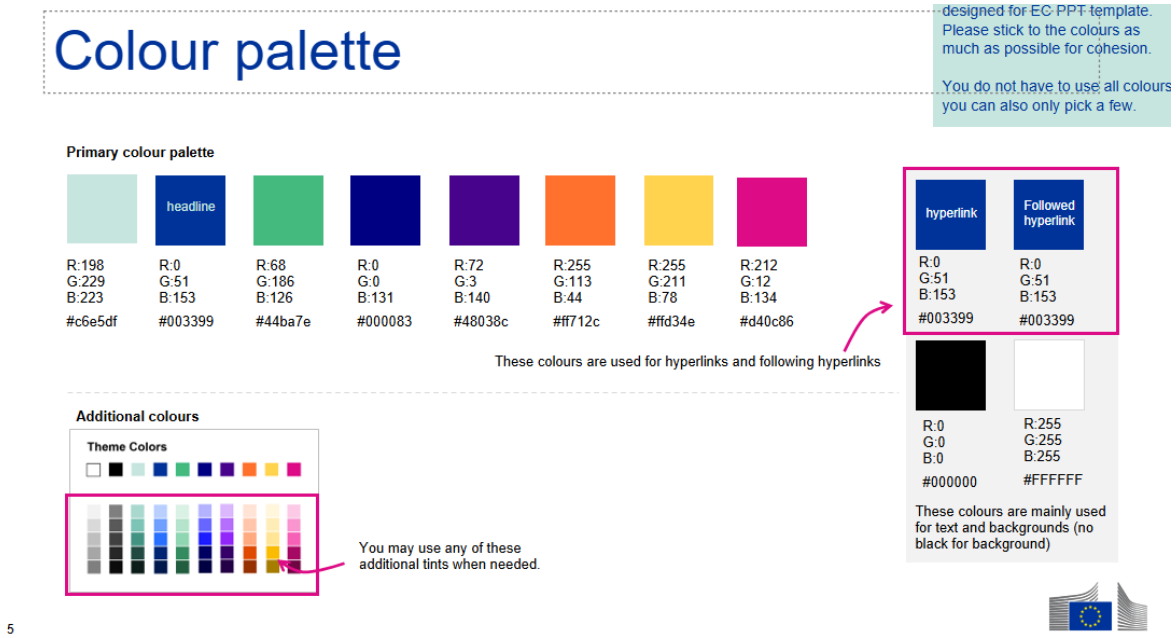
Reference List

- Crestron Electronics. (n.d.). Room scheduling solutions. Crestron. https://www.crestron.com/Products/Featured-Solutions/Room-Scheduling
- AskCody. (n.d.). Meeting room management. AskCody.
- https://help.askcody.com/getting-started-with-meeting-room-displays
- World Wide Web Consortium. (2018, June 5). Web Content Accessibility Guidelines (WCAG) 2.1. W3C. https://www.w3.org/TR/WCAG21/
- European Commission – DG SCIC. (2026, April). IBBDiS internal documentation [Internal document].
- European Commission. (n.d.). EC visual identity guidelines. European Commission. https://commission.europa.eu/resources/european-commission-visual-identity_en
- European Commission – DG SCIC. (2026). MERASE ecosystem documentation [Internal document].

Attachments

Attachment 1 — EC Colour Palette

Colour palette from the EC PowerPoint template guidelines, used as the visual reference throughout the project.



AI Usage Declaration

How it was used

During this internship, AI tools were used as a support throughout the project. They were mainly used for drafting and improving written content such as messages, documentation, and sections of this realization document, as well as for getting explanations of technical concepts and troubleshooting issues encountered during development.

Limitations and Verification

AI-generated content was never used directly without review. Everything was rewritten in my own words before being delivered. All technical decisions, the design choices, and reflections in this document are from the work that I have and the experiences from the internship. The AI was used as a writing and explanation tool, not as a replacement for my own thinking and judgment.